

Agent 工具與協同工作能力

5-Day AI Agents – Intensive Vibe Coding Course With Google

Kanchana Patlolla, Lukasz Olejniczak, and Pier Paolo Ippolito

目錄

前言	2
為何撰寫本文，為何是現在	4
本文適用對象	4
講求氛圍的程式開發者眼中的 MCP：探索、設定與連線	5
探索	6
組態設定	7
連線	7
繞過 NxM 原型開發問題	7
為何這很重要？	9
MCP 伺服器的除錯問題	9
Vibe Coder 工具組：MCP 使用的最佳實踐	10
代理架構的演進	11
建立虛擬工作力	17
實作 A2A 協定	18
A2A Agent 的商業化	21
溝通鴻溝	23
生成式 UI 與 A2UI	23
產生 A2UI：兩種模式	26
互動式產物與 Canvas	28
最佳實踐	29
摘要	32
Agent 與商務 (AP2 與 UCP)	33
具備工作流程範例的自主採購	33
協定的主要特性與優勢	33
AP2 與 UCP 是什麼？	34
結論	36

作者：Kanchana Patlolla, Łukasz Olejniczak, 與 Pier Paolo Ippolito Alan Blount

Mike Smith

Anant Nawalgaria

Lukas Geiger

Anant Nawalgaria

設計師

Michael Lanning

前言

在 Day 1 中，我們概述了從傳統軟體開發到 Agentic 工程的典範轉移，並介紹了工廠模型，在這個模型中，開發者的主要產出不再是原始語法，而是生成程式碼的系統。我們定義了此系統的核心架構：

如果 Agentic 工程代表你正在協調的工廠車間，那麼 **MCP**、**A2A**、**A2UI**、**AP2** 和 **UCP** 就是**產業標準**——統一的螺母、螺栓和螺絲。

大小、資料格式與通訊通道——這些讓你的機械能夠安全地與外界互動。

若缺乏這些開放協定，你建立的每個 Agent 都只能像車庫裡的「客製化機器」般孤立存在。你被迫耗費大量時間與 Token，為每個工具和 API 連結撰寫脆弱的客製化封裝程式，將自己困在低效的 **Conductor** 角色中。

透過採用標準化的互操作性層，你能將 Agent 的 **Harness** 轉化為模組化、隨插即用的平台。你將減少在除錯自訂 JSON 載荷上的時間，轉而以 **Orchestrator** 的身分引導高層次意圖。

- **OpenResponses & Interactions API** 均為「**Power Plugs**」，是支援長時間執行任務的現代 LLM 推理 API 方法。這些方法模糊了無狀態單輪互動與有狀態 Agent 之間的界線。
- **MCP (Model Context Protocol)** 在 Agent 的 **Harness** 中扮演「**USB-C**」的角色，能即時將模型連接至資料庫、檔案系統與網路 API。
- **Skills** 是「**Playbooks**」，即可在沙盒環境（如終端機）中執行的簡單 Markdown 指令、腳本或工具。

- A2A (Agent-to-Agent) 如同「Factory Radio」，讓專業化的 Agent 能相互協商、集思廣益並委派任務。
- A2UI (Agent-to-User Interface) 如同「Generative Display Window」，將原始、複雜的 JSON 輸出轉化為安全、互動式的視覺化元件，供人類操作員使用。
- AP2 與 UCP 扮演「Global Supply Chain & Transaction Network」的角色，讓 Agent 能安全地協商並執行自主商業交易。

本文聚焦於「氛圍編碼者」(vibe coder) 如何在短短一個下午內，快速運用這些協定建構虛擬資料與執行團隊。



圖 1：Agent 協定生態系

為何撰寫本文，為何是現在

在結構較少的「氛圍編碼」(vibecoding) 時代，Harness 與協定有助於建立對人工智慧系統的信任。

在 Agent 開發流程中，雖然成果的速度仍是開發者與實務工作者的首要驅動力，但標準化協定能讓我們在實現複雜目標時走得更遠，將孤立的「客製化機器」轉化為模組化、可互操作的平台。若缺乏共識的開放標準，開發者將不斷累積技術負債，每個 API 都成為獨一無二的標準。這些都是低槓桿的工作——為每個工具撰寫脆弱的客製化封裝、長期維護並適應他人需求。採用這些分層架構後，角色將從單純的建構者轉變為高階協調者。

本文適用對象

本文專為軟體工程師、工程經理、架構師及技術領導者設計，他們認識到邁向 Agentic Engineering 的轉變需要嚴格遵循協定，以確保成果的精確性與可靠性。本文同時作為「直覺式程式設計師」(vibe coders) 的實用指南，這些開發者重視速度與視覺化成果，並展示如何建構一個虛擬的資料與執行團隊。

應用小技巧 (Applied Tip)：

在《Day 1》白皮書中，我們建議使用 AGENTS.MD 檔案來提供 Agent 程式設計的標準指引。

開始撰寫程式前務必深思熟慮——明確陳述你的假設、揭示權衡取捨，並在遇到模糊不清的情況時立即停下來尋求釐清，切勿默默猜測。

只撰寫解決當前問題所需的最少程式碼，嚴格避免預測性功能、未經要求的抽象化或預設配置。修改現有程式碼時，應進行高度精準的修改，僅更新完成任務所需的確切程式碼行，完美保持既有風格，並確保未受影響的相鄰程式碼完全不受干擾——除非你的變更直接導致某個匯入或變數成為孤立狀態。

最後，**以目標驅動的方式執行每項任務**，將其分解為清晰的步驟計畫，並設定明確的成功標準，例如先撰寫可重現或失敗的測試，並獨立進行驗證迴圈，直到該特定目標嚴格達成為止。

在我們先前的白皮書《Agent 工具與互操作性：Model Context Protocol (MCP)》中，我們闡述了 MCP 的企業架構，詳細說明主機-客戶端-伺服器拓撲、自訂伺服器建立及安全

治理。

然而，對於講求氛圍的程式開發者來說，**消費優先於建立**。你不會想花費數小時撰寫自訂伺服器設定，而是希望能夠連結現有的公開與私有登錄庫，立即為你的 Agent 賦予「隨插即用」的超能力。本節將介紹如何有效率地消費 MCP 伺服器、如何繞過 NxM 整合危機，以及當傳輸層出現問題時如何進行除錯。

注意事項 (Note)：

我們將在下一份白皮書中深入探討 Agent Skills，並在後續文件中討論安全性。

講求氛圍的程式開發者眼中的 MCP：探索、設定與連線

傳統企業將工具整合至 LLM 的方式是**客製化硬接線**。每個新連線都需要自訂 REST 包裝器、API 金鑰、Token 更新政策以及自訂 JSON 解析器。MCP 以標準化的通訊端取代了這些摩擦。設定你的程式開發 Agent 僅需三個簡單步驟：

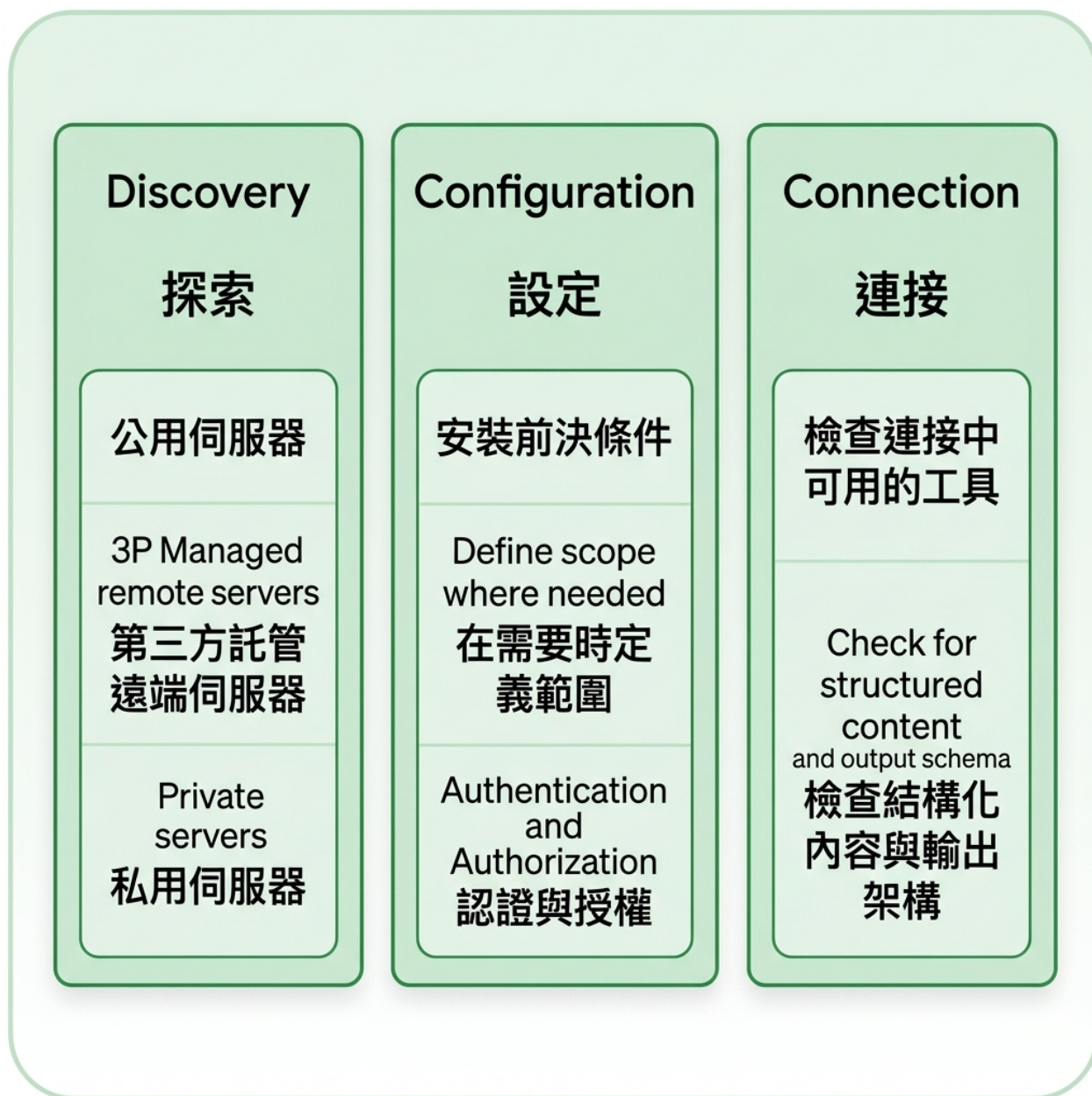


圖 2：MCP 伺服器上線步驟

探索

講求氛圍的程式開發者不會撰寫自訂連接器，而是從三個主要來源尋找預先建立的 MCP 伺服器：

- **公開 MCP 登錄庫**：公開可用的登錄庫託管了數百個預先建立的伺服器（例如：registry.modelcontextprotocol.io、github.com/mcp）。這些資源非常適合快速進行本

地原型開發，但未經審核，使用風險自負。

- **第三方 (3P) 遠端 MCP 伺服器**：經過審核的平台提供託管端點。從官方來源探索經過審核的 MCP 伺服器。例如，Google 官方發布的 MCP 伺服器（如 Google Maps、BigQuery、Google Docs）讓你的 Agent 能夠安全地與託管服務互動。
- **內部登錄庫**：在組織內部，內部工具會以 MCP 伺服器形式公開，並收錄於安全的登錄庫中。此登錄庫的技術實作通常透過 API Gateway、GCP Agent Registry 或私有微服務入口管理。

應用小技巧 (Applied Tip)：

在做出決策時，務必將安全性列為首要考量，優先尋找官方支援的 MCP 伺服器。

如有可能，請遵循以下指示。

若使用公開社群伺服器，請勿傳遞憑證，建議使用如 **Model Armor** 等服務以避免安全性問題。

組態設定

在連線之前，請指定標準參數以詳細說明伺服器的範圍與權限。這包括建立環境檔案來處理 API 憑證（如個人存取權杖或 OAuth 密鑰），並定義檔案系統的讀寫權限以保護資料。

應用小技巧 (Applied Tip)：

檢查前置需求 確認範圍與存取條件 將規格納入程式碼 Agent 驗證

連線

完成組態設定後，用戶端將與伺服器建立連線。為驗證連線狀態，主機用戶端會執行基本的握手請求，列出可用的工具並驗證輸出結構描述。

繞過 NxM 原型開發問題

為了理解 MCP 為何對快速原型開發至關重要，讓我們來看整合數學。如果你正在嘗試 N 個不同的 LLM（Gemini 3.1 Pro、Gemini 3 Flash、Gemini 3.5 Flash 或本地開源模型），

並希望將它們連接到 M 個不同的外部工具（Jira、BigQuery、GitHub、Google Drive）。傳統的臨時開發方式需要為每個模型與工具的交集撰寫自訂整合程式碼：

$$\{\text{整合複雜度}\} = O(N \times M)$$

如果你有 5 個模型和 10 個工具，就必須維護 **50 個客製化整合點**。如果單一工具的 API 發生變更，多個解析迴圈將在缺乏協定的情況下失效。

傳統整合方式：

模型 工具

[Model A] — [Tool 1]
[Tool 2]
[Model B] — [Tool 2]
[Model C] — [Tool 3]

MCP 互通性：

模型 MCP 工具

[Model A] — [Tool 1] [Model B] — [Tool 2]
[Model C] — [Tool 3] [MCP] — [Tool 1] [MCP] — [Tool 2] [MCP] — [Tool 3]
工作量：O(N × M) 工作量：O(N + M)

程式碼片段 1：MCP 複雜度

MCP 將此複雜度降低至線性規模：

$$\{\text{整合複雜度}\} = O(N + M)$$

為何這很重要？

透過標準化工具定義，你可以直接將標準傳輸協定整合到 Agent 的 Harness 中，而無需撰寫整合程式碼：

- **stdio (標準輸入/輸出)**：最常用於本地與原型開發。你的程式碼 Agent 可以在無需複雜網路連線設定的情況下執行，將工具視為本地程序。主機用戶端會以本地背景子程序啟動 MCP 伺服器，透過 stdin 和 stdout 傳遞 JSON-RPC 2.0 訊息。
- **SSE (Server-Sent Events) over HTTP**：主機用戶端（可以是本地或已部署的 Agent 應用程式）透過標準網路協定連接到遠端 MCP 端點，即時串流資料至 Agent。這種方式相較於 Studio 方法具有多項優勢，包括更少的依賴性、始終保持最新狀態、佔用空間更小且生命週期更簡單，但對雲端託管的 MCP 伺服器負擔較大。

這兩種選項讓開發者能夠在無需多個自訂整合層的情況下採用該平台。

MCP 伺服器的除錯問題

當你的 Agent 出現參數幻覺 (hallucinates parameters)、呼叫錯誤的工具或無法解析酬載 (payload) 時，不要浪費時間盲目修改系統指令。請直接對傳輸管道進行除錯：

- **MCP Inspector**：一款原生開發者工具，可執行本機 Web 面板。它允許你手動查詢任

何本機或遠端的 MCP 伺服器、檢視目前的工具結構描述 (tool schemas)、手動測試酬載輸入，並檢查原始的 JSON-RPC 2.0 封包，而無需啟動主要的 Agent 工作流程。

- **Chrome DevTools**：適用於執行 Web 開發環境或除錯 SSE 連線時，可追蹤傳入的 Web 資料流並檢查伺服器延遲。

Vibe Coder 工具組：MCP 使用的最佳實踐

為了在維持速度的同時不犧牲穩定性，Vibe Coder 在使用 MCP 伺服器時應遵循以下準則：

應遵循的事項（最佳實踐）

- **審核公開伺服器後再連線**：在將公開的開源 MCP 伺服器連接到可存取本機檔案系統或憑證的 Agent 前，務必先檢視其程式碼。
- **使用 RAG 管理工具**：保持 Agent 的上下文視窗 (context window) 整潔。僅在需要時從註冊表動態載入工具，並在任務完成後將其從上下文中移除，以避免注意力分散。
- **善用內部 API 閘道與註冊表**：如有可能，應依賴內部工具註冊表。這能確保你使用的是經過核准與治理的資料結構描述 (data schemas)，而非重新發明輪子或執行未經審核的程式碼。
- **使用 MCP Inspector**：當 Agent 出現工具呼叫參數幻覺時，請使用 MCP Inspector 或 Chrome DevTools 檢視原始傳輸資料，而非僅依賴系統指令調整。
- **務必納入人工介入 (HITL)**：在呼叫伺服器前向使用者顯示工具輸入，避免惡意或意外的資料外洩。
- **務必滿足稽核需求**：記錄工具使用情況以供稽核。

禁忌（常見陷阱）

- **不要重複造輪子**：避免撰寫自訂的 REST API 包裝程式。優先搜尋現有的 MCP 伺服器，以維持「通用插座」的設計哲學。
- **不要在正式環境中使用公開且未驗證的 MCP**：開源 MCP 伺服器非常適合週末原型開發，但存在安全性與可靠性風險。切勿將核心業務邏輯綁定於未驗證的公開端點或 API 包裝程式碼。

- **不要硬編碼憑證：**在設定階段，切勿將 API 金鑰或驗證 Token 直接貼入 Agent 的 Prompt 或本地腳本中。應透過環境變數將憑證傳遞給 MCP 伺服器。
- **不要連接正式環境：**使用 MCP 伺服器時應搭配開發專案，而非正式環境。大型語言模型（LLM）非常適合協助設計與測試應用程式，因此應在安全的環境中善加利用，避免暴露真實資料。確保開發環境中的資料為非正式資料（或經過混淆處理）。
- **不要用於更新操作：**若必須連接真實資料，請將伺服器設為唯讀模式，確保所有查詢均以唯讀方式執行。
- **不要提供對所有專案的廣泛存取權限：**將 MCP 伺服器的範圍限縮至特定專案，僅在必要時開放該專案的資源存取權限。

我們已學習如何讓 Vibe 開發者善用 MCP 工具來建立 AI Agent，但 AI Agent 的建立不僅限於工具，更在於實現使用者在應用程式生態系統中的互通性。接下來我們將探討 Agent 互通性的運作方式。

隨著 AI 系統逐步演進為由專業領域專家組成的分散式網路，標準化通訊對於擴展規模變得至關重要。本節將探討 **Agent-to-Agent (A2A)** 協定作為解決生態系統碎片化的基礎層，使開發者能夠探索、協調並變現全球互通的虛擬工作力。

代理架構的演進

為了理解 **Vibe Coding** 在代理架構演進中的定位，我們可以觀察到技術發展的一個重複模式：從手動、低階的組態設定，逐步轉向高階、基於意圖的宣告式協調，讓使用者不再需要說明「如何」建立，而是直接表達「需要什麼」。

IT 基礎架構的歷史中也能看到類似的趨勢，從手動組態轉向「基礎架構即程式碼 (Infrastructure as Code)」的宣告式腳本。機器學習的發展歷程中也出現了相同的趨勢。Google 在自動化機器學習 (AutoML) 背後的願景，是透過自動化模型建立的「繁重工作」來普及人工智慧：

技術願景：

「我們希望讓人工智慧更易於存取的其中一種方式，是簡化機器學習模型（稱為神經網路）的建立過程。如今，設計神經網路極為耗時……這就是我們開發 AutoML 的原因，證明神經網路能夠設計神經網路。我們希望 AutoML 能將目前僅有少數博士擁有的能力……」 [1]

如今，技術讓我們能夠透過 **Vibe Coding** 建立整個應用程式。然而，用於 **Vibe Coding** 的代理架構發展軌跡，正在重現軟體史上最重大的轉變之一：**從單體式架構 (Monolithic) 轉向微服務架構 (Microservices)**。

正如早期的網路應用程式是以單一龐大的程式碼庫建立，其中每個功能（計費、使用者介面、資料庫）都緊密耦合，許多初期的 **Vibe Coding** 專案也依賴於「**瑞士刀式單一代理單體 (Swiss Army knife Single Agent Monolith)**」。這種模式依賴單一高度複雜的 Prompt，讓一個代理身兼多職，連接多種工具來處理從資料庫查詢到使用者介面渲染與測試的所有工作。雖然這讓開發者能在一個週末內快速建立原型，但最終會遇到與早期網路應用程式相同的「單體式天花板」問題：

- **擴展摩擦**：無法單獨優化「資料庫邏輯」而不影響位於同一個 **Prompt** 中的「UI 邏輯」。此外，單一 **Agent** 可存取的工具越多，其決策能力越差。因為下一步動作的搜尋空間過於龐大，容易導致參數幻覺或觸發錯誤的工具。
- **上下文超載**：將全局系統指令、數十個複雜的工具架構以及持續對話歷史全部塞入單一 **Prompt**，很快就會耗盡模型的工作記憶體。
- **單點故障**：單一「工具」或指令中的錯誤可能導致整個 **Agent** 產生幻覺或崩潰。無關或損壞的資料若儲存在上下文中，會持續影響後續推理。

為了克服這些瓶頸，可以參考機器學習工程師與軟體工程師多年前制定的藍圖。當早期 AutoML 模型成功驗證其商業價值時，團隊鮮少讓這些自動生成的黑盒直接支撐核心業務。為了達到生產規模，他們必須將黑盒拆解為可觀測、可維護的階段——包括資料版本控制、特徵儲存、漂移偵測等。透過專精化各個元件，他們應用了系統設計的基本法則：**專精化作為擴展機制**。

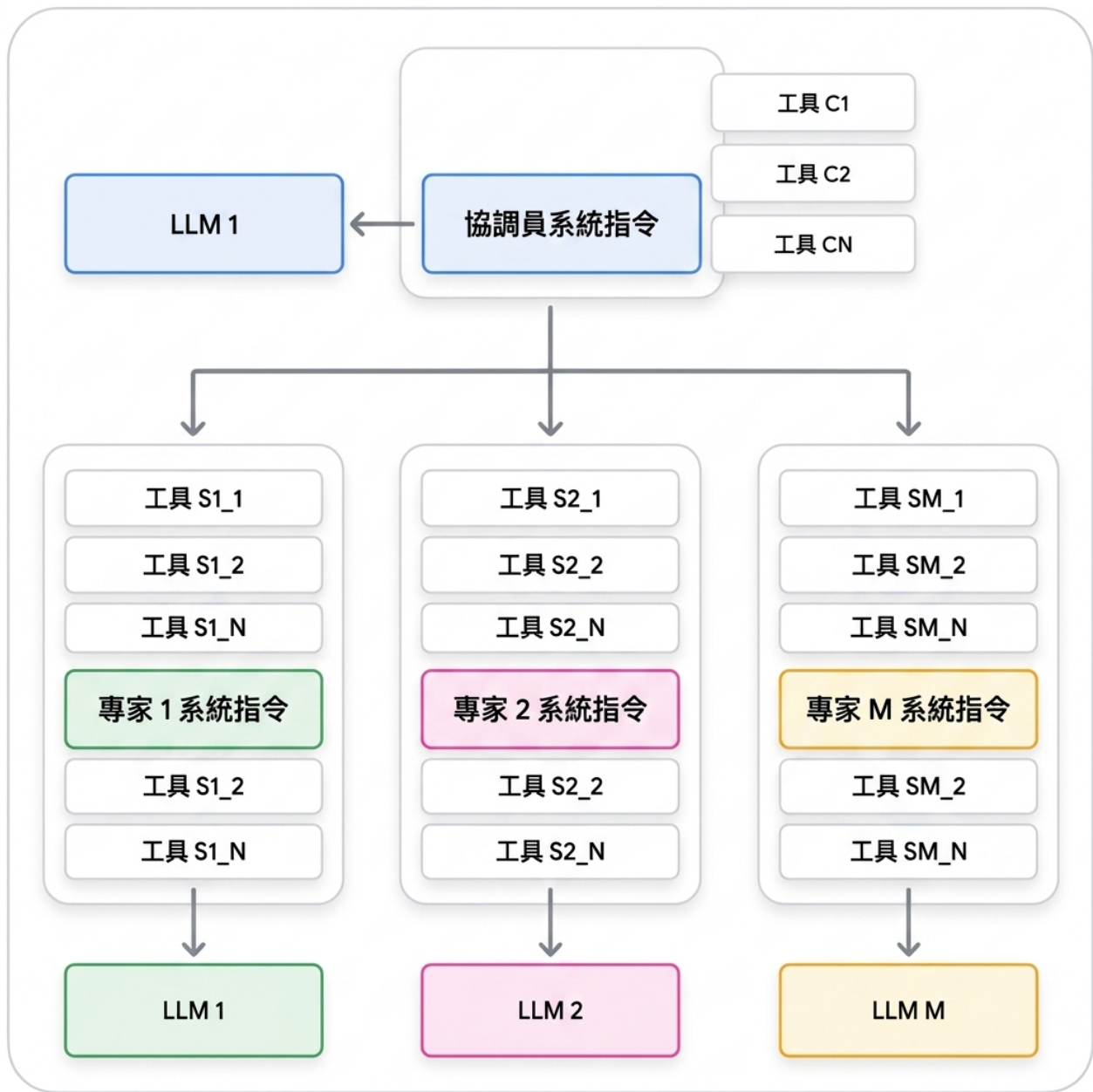


圖 3：單體式多智慧體架構

內部專精化透過邏輯分區，將單體式 **Agent** 拆解為多個具備明確目的的子 **Agent**。每個子 **Agent** 由高度聚焦的系統 **Prompt** 及相關工具子集所管理。雖然仍屬單體架構（因為這些內部子 **Agent** 不跨網路邊界通訊），但它們共享相同的執行環境與底層記憶體。

此邏輯分區讓開發者能模組化複雜定義，同時保留單一程序應用程式固有的低延遲執行與簡化狀態管理優勢。同時，這種模組化設計也解決了單體 **Agent** 的固有限制。

透過多項關鍵技術改進，單一 **Agent** 系統得以實現：

- **搜尋空間縮減**：透過限制子 Agent 的工具與技能（例如僅提供資料庫 Agent 查詢工具），能大幅降低工具呼叫錯誤與幻覺（hallucinations）問題。
- **注意力稀釋緩解**：專業化使底層 LLM 能專注於單一領域的 Prompt，進而提升推理精確度。
- **上下文負載優化**：由於協調器（orchestrator）負責任務路由而非處理整個邏輯樹，子 Agent 的上下文視窗能維持高訊噪比。

隨著 Agent 架構的成熟，基於分散式多 Agent 架構的生態系統轉變正在進行。包括 Google、Salesforce、ServiceNow 與 Workday 在內的產業領導者，已不再僅提供 API，而是部署專為其專屬生態系統設計的領域特定 AI Agent，以實現原生精準度。

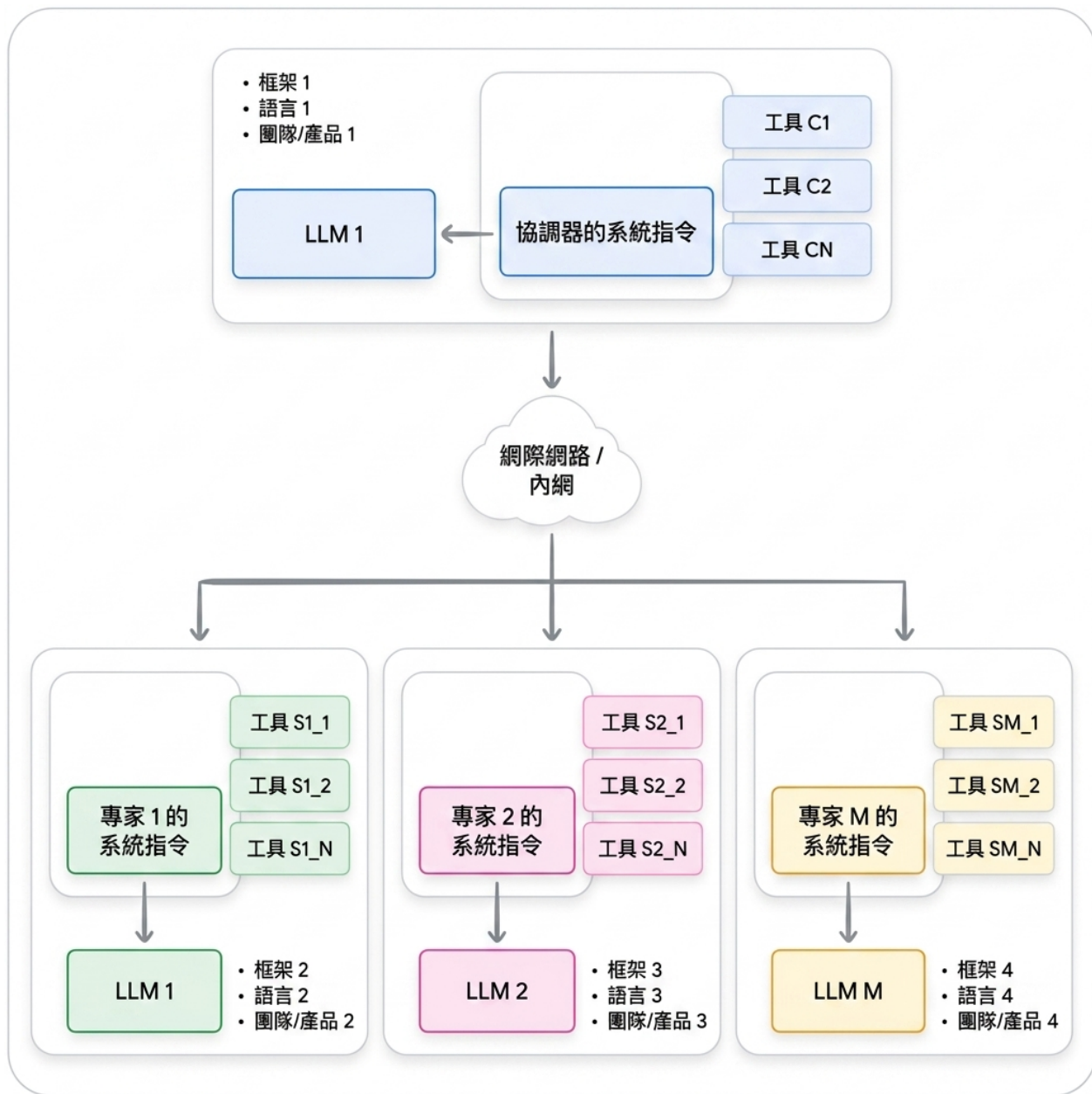


圖 4：分散式多 Agent 架構。協調器透過網路邊界將任務委派給遠端的領域特定 Agent。此一轉變為開發者帶來策略性機會。然而，為了達到下一階段的規模化，技術與業務領導者必須透過現代「自建 vs. 購買」的視角，嚴格優先排定重點。

雖然技術上可行，但透過自建子 Agent 來構建多 Agent 系統（例如為 ServiceNow 或 Salesforce 等第三方平台建立自訂子 Agent），此作法將帶來顯著的維護成本。選擇自行開發領域專家 Agent，開發者需自行負責更新 Prompt 邏輯與工具定義，以反映上游產品更新與 API 架構變更。

更優的架構策略是建立高價值應用時，應善用官方專家 Agent。這些 Agent 由具備深厚

領域專業知識的團隊維護，確保最高的可靠性與效能。透過將專業領域邏輯的維護工作交由這些官方實體處理，**協調器（Orchestrator）**——以及延伸至開發者——便能專注於應用程式的獨特使用者價值與核心創新。

然而，試圖協調這個由分散式人工智慧專家組成的虛擬團隊，卻引入了新的瓶頸：**碎片化（fragmentation）**。這些專家代理（Agent）可能由不同團隊建立，並使用不同的技術。Google 的代理可能使用 Python、Go 或 Java 搭配 ADK 編寫，Salesforce 的代理可能基於 LangChain 框架運行，而 Workday 的代理可能使用完全客製化的解決方案。這些代理存在於我們的網路邊界之外，使用不同的語言、期望完全不同的酬載（payload）結構、以不同方式處理對話狀態，並依賴各異的傳輸層。

如果開發者必須為每個想要整合的專家撰寫客製化的整合程式碼，並處理各自的錯誤修正迴圈，那麼「虛擬團隊」概念將瞬間變成一項整合工程。維護這些客製化橋接器不崩潰的成本將吞噬整個專案。

這種碎片化的混亂正是 **代理對代理（Agent-to-Agent, A2A）協定 [A2A]** 所設計要標準化的問題。A2A 最初由 Google 開發，現已捐贈給 Linux 基金會，為代理系統引入了一個通用的通訊層。它作為人工智慧生態系的共通語言，抽象化了網路傳輸的細節、底層框架、程式語言以及酬載的差異。

它確保中央協調器能夠探索、導入並與生態系中的任何專家代理協作，完全無需關心該專家代理的底層實作方式。正如 HTTP 標準化了網路，**A2A 標準化了虛擬工作力**。

「呼叫者需要的是結果，還是需要另一個參與者承擔責任？」——這是我見過對此決策最簡潔的框架。中央代理提示（Prompt）不小心演變成工作流引擎的徵兆，正是大多數團隊在三個月後才發現需要協作語義的原因。[19]

但這引出了下一個架構問題：如果我們只是將任務委派給外部專家，為何不能將其視為標準工具？

與專家和工具的互動本質上存在根本差異。試想：

一位正在翻修廚房的屋主面臨抉擇：是購買單獨的工具和手冊，嘗試自己動手施工，還是將專案委託給專門從事廚房建造的領域專家。

工具只是被動的器具；專家則是協作夥伴。當你聘請專家時，不會只是交給他一份藍圖就離開，期待完美的廚房會憑空出現。專家可能會遇到設計上的邊界情況或疏漏，需要暫停工作、與你討論取捨，然後再繼續。

有界 vs. 無界領域 這正是將專家級 AI Agent 視為簡單工具無法擴展的原因之一。標準工具或 API 以「發射後不管」(fire-and-forget) 機制運作，期望接收單一、格式完美的請求（藍圖酬載），然後返回回應。然而，真實世界的軟體環境往往存在數位版的「歪斜牆面」：模糊的資料結構、誤導性的需求和衝突的使用者偏好。

然而，Agent 運作於無界的問題解決空間。真實世界的軟體環境充滿模糊的資料結構、誤導性的需求和衝突的使用者偏好。在複雜的工作流程中，你很少能在不經多輪釐清的情況下，就指定所有必要的細節——這正是數位版的「歪斜牆面」。

Agent 架構中的 GOTO 問題 由於 Agent 的領域是無界的，強行將 Agent 塞入標準工具封裝，會在協調器中引入架構層面的 GOTO 敘述。

當你呼叫協作型 Agent 時，控制流會脫離預期的結構化情境。Agent 可能會進入中斷狀態、要求更多資訊，甚至在使用者改變主意或放棄 Prompt 的情況下，永遠無法將預期的輸出返回給原始呼叫者。

你需要一種能隔離這種混亂、多輪狀態的範式。你需要一個協定，允許領域 Agent 暫停執行、回溯至協調器 (Orchestrator)、協商解決方案，然後在不遺失對話狀態的情況下恢復工作。

這正是 **A2A 協定** 填補的缺口。透過將協作路由隔離，

在 A2A 層，我們保持工具層 (MCP) 的乾淨、可預測性與嚴格結構化。

建立虛擬工作力

A2A 協定的引入與專業化轉向不僅解決了技術瓶頸，更為專業知識建立了新的市場基礎，代表價值交付模式的全新典範。

若缺乏 A2A，開發者在建立應用程式時可能會因日益增加的複雜性而疲於應對。在 A2A 時代，同一位開發者能專注於打造高價值的利基領域——例如專精於「即時法規遵循」的 Agent。無論是建構為複雜的多 Agent 單體架構，還是分散式應用程式，這些系統如今都能以符合 A2A 標準的 Agent 形式對外開放。這意味著，在世界某個角落編寫的專家級 Agent，能被全球另一端的 Orchestrator 發現並「聘用」。

Agent Card 為促進這種發現機制，每個 Agent 都由一份 **Agent Card** 定義。這是人工智慧世界的標準化「履歷」，提供機器可讀的身份識別，概述以下內容：

- **能力**：Agent 能執行的任務。
- **安全性與合規性**：資料處理政策與權限需求。
- **互動架構**：其他 Agent 透過 A2A 協定與其通訊的方式。

註冊中心：從公共市集到私有企業 當 Agent 配備了 Agent Card 後，即可註冊至 **Agent Registry**，其目標是將 Agent 轉化為可被發現的服務。這為開發者提供了兩種主要路徑：

1. 公共註冊中心（市集）：如同全球人才仲介，公共註冊中心讓開發者能將其專家級 Agent 公開上架，供全球搜尋。這為特定產業的專家級 Agent 鋪平道路，並能將其專業知識授權給數千個其他 Orchestrator 使用。

2. 私有登錄中心：在企業內部，Agent 登錄中心 提供了一個安全、受管控的環境。大型企業內的開發者可以建立專門的 Agent 來自動化內部工作流程，並將其註冊供其他部門使用，確保專業知識能夠跨越企業邊界共享，同時不影響安全性。

最終，A2A 協定將建立孤立的 Agent 應用程式轉變為建立 **全球互通數位工作力的基礎成員**。

實作 A2A 協定

A2A 架構的實際應用支援兩種不同的開發模式：

- 將 AI Agent 暴露為 A2A Agent（供給端）
- 協調遠端 A2A Agent（需求端）

暴露 A2A Agent 將 Agent 封裝至 A2A 生態系需經過三個核心步驟：

- **定義 Agent 卡片**：正式的 Agent 規格。
- **實作 Agent 執行器（翻譯層）**：將傳入的 A2A 請求與回應轉譯為底層 Agent 框架（如 ADK、LangGraph 或企業自訂程式碼）所需的特定呼叫。
- **建立 A2A 端點**：執行器必須暴露為符合 A2A 標準的端點。

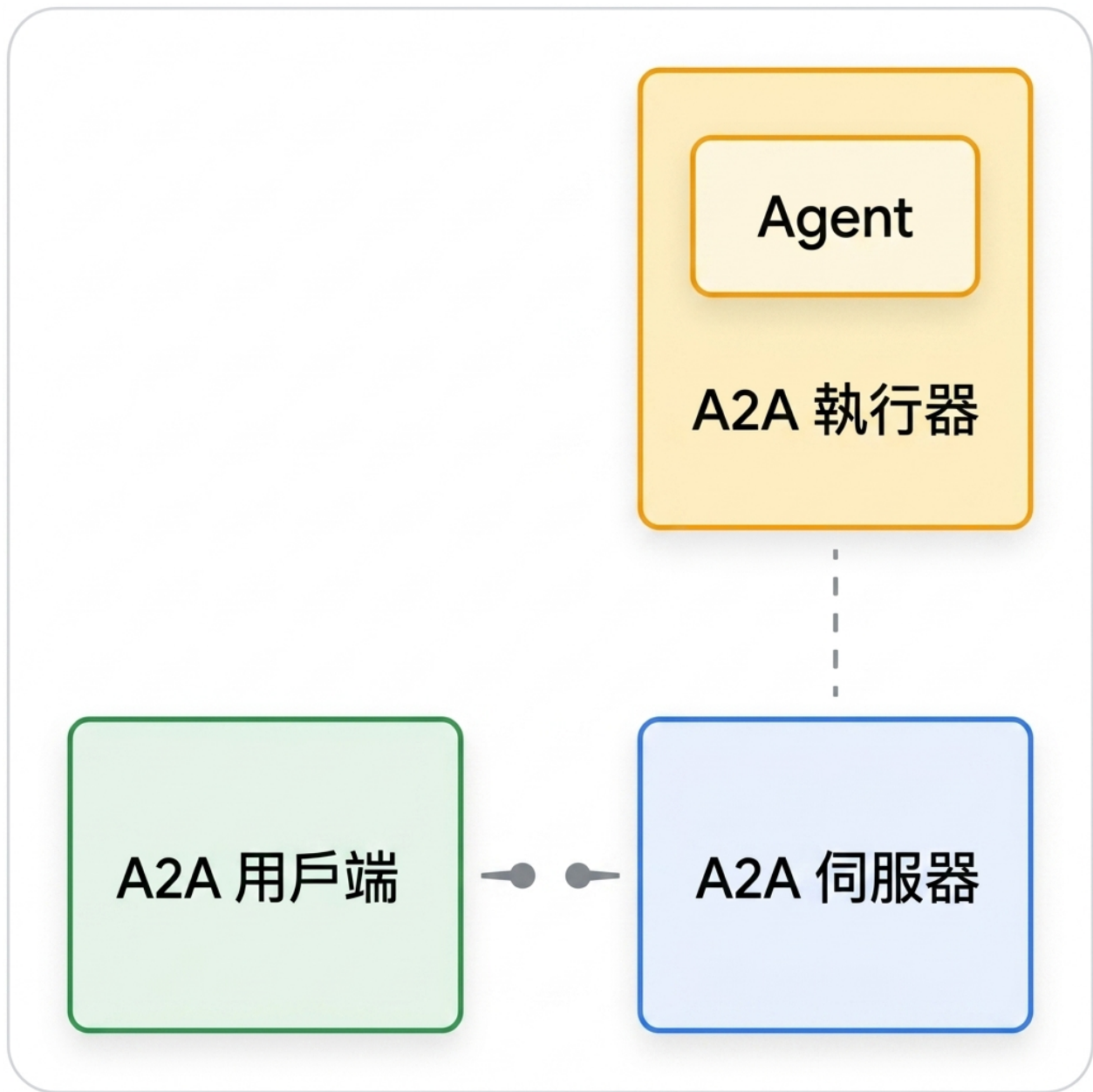


圖 5：原生 AI Agent 以 A2A 伺服器形式暴露於供給端，並透過 A2A 用戶端於需求端消費。

連接遠端 A2A Agent 在成熟的生態系中，應用程式不需原生具備對其涉及的每個領域的深入知識。相反，它扮演 **協調器 (Orchestrator)** 的角色——一個核心樞紐，其主要認知負荷專注於理解使用者意圖、管理整體工作流程，並將特定任務委派給專業的遠端 A2A Agent。

遠端 A2A Agent 以自主、特定領域的承包商角色運作，並透過 A2A 協定進行通訊。連接

這些遠端 Agent 通常遵循以下兩種架構模式之一（程式碼範例來自 Google ADK）：直接點對點整合：

Python

```
# 1. 透過硬編碼端點直接實例化

# 適用於特定廠商整合或私有 Agent
billing_specialist = RemoteA2aAgent(
    name="billing_agent",
    endpoint="https://api.vendor.com/v1/billing/a2a"
)
```

程式碼片段 2：透過硬編碼端點直接實例化遠端 A2A Agent。

或透過 Agent 註冊表來探索 Agent：

Python

```
# 2. 透過 Agent 註冊表間接實例化

registry = AgentRegistry(project_id=project_id, location=location)

# 註冊表會解析資源名稱並處理驗證
agent_name = f"projects/{project_id}/locations/{location}/agents/YOUR_AGENT_ID"
my_remote_agent = registry.get_remote_a2a_agent(agent_name=agent_name)
```

程式碼片段 3：透過 Agent 註冊表間接探索並實例化遠端 A2A Agent。

透過解決早期人工智慧生態系統中固有的碎片化問題，A2A 協定建立了一個統一的通訊層，進而讓開發者能在此標準化基礎上進行建構。雖然 A2A 協定核心作為傳輸與協商的骨幹，但實現豐富的交易型應用程式通常需要高度專業化的功能。

這一需求透過 A2A 擴充功能 [1] 機制來滿足。它提供了一種標準化模式，讓 Agent 能安全地宣告、協商並執行可選的高階功能，這些功能超越了基本的訊息傳遞。

在核心 A2A 基礎上運作的三個基礎框架擴充套件，包括 **Agent-to-User Interface (A2UI)**，旨在產生動態、有狀態的使用者體驗；**Universal Commerce Protocol (UCP)**，專為促進安全、自主的代理商務而設計；以及 **Agent Payments Protocol (AP2)**，為可

信任、可驗證的代理支付奠定基礎。這三者將在以下章節中進一步探討。

A2A Agent 的商業化

讓 Agent 運作起來只是成功的一半，另一半則是確保長期商業可持續性。借鑑軟體即服務 (SaaS) 模式的成功經驗，A2A 協定自然支援 **Agent-as-a-Service (AaaS)** 模式。這使得專業 Agent 能透過各種銷售管道，以消費型模式提供服務。

其中一個範例是 **利用 Google Cloud Marketplace 作為商業化引擎**。獨立的 Agent/軟體供應商與開發者可以在 Marketplace 上架其 A2A Agent，立即觸及 Google Cloud 現有的企業客戶群。客戶能在採購這些專業 Agent 的同時，利用其現有的 GCP 財務承諾，實現多方共贏。

Marketplace 基礎架構自動處理「棘手的部分」——複雜的計費流程——透過原生支援混合定價模式，例如「**固定費用加使用量**」模式。這讓供應商能收取可預測的基本費用，同時針對運算或 Token 超額使用進行商業化。

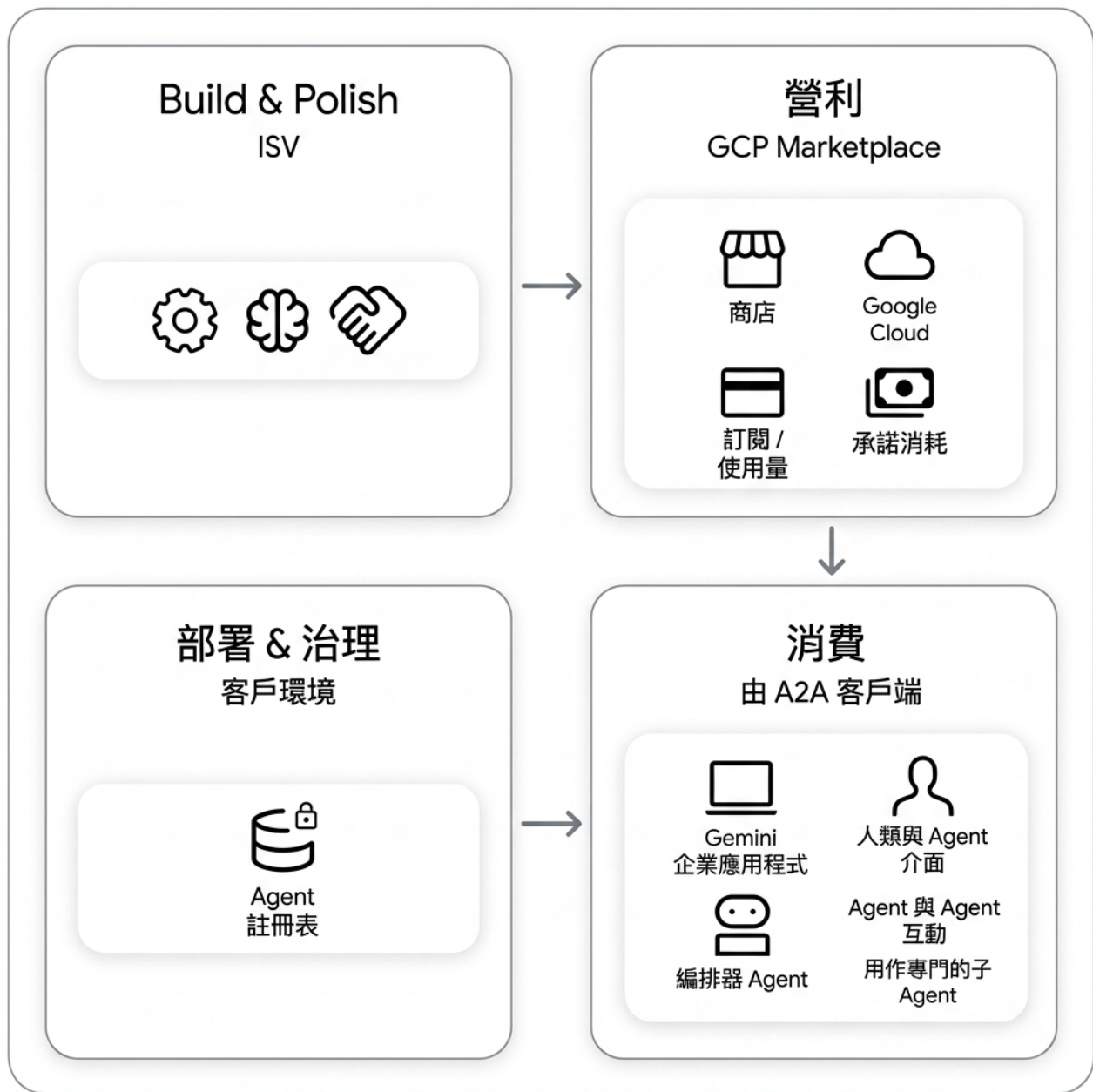


圖 6：Agent-as-a-Service (AaaS) 生命週期。

在 Google Cloud Marketplace 上註冊的 Agent 可直接供 **Gemini Enterprise** 應用程式使用者存取。Gemini Enterprise 是一個先進的代理平台，將 Google AI 的最佳功能帶給每位員工，應用於各種工作流程。它讓團隊能在單一安全環境中探索、建立、分享及執行 AI Agent。透過與 Agent 登錄中心整合，並作為原生 **A2A Client** 運作，該平台透過提供員工廣泛的專業工作者生態系統，增強人類的工作體驗。同時，Gemini Enterprise 也是 **Agent as a Service** 平台（透過其 AaaS 功能）與 **Agent as a Product** 的典型範例。

Assistant API)，同時也是遠端 Agent 的主機。

雖然雲端市集與傳統付款閘道將處理大部分的 Agent-as-a-Service (AaaS) 計費，A2A 協定也支援無需許可的機器對機器微交易，讓開發者能完全避免管理使用者帳戶。

透過使用 A2A Extensions 框架 [A2AEXT]，Agent 的伺服器可以實作 x402 (或 L402) 標準。在此模式下，伺服器會攔截請求，若未付款則回傳 HTTP 402 Payment Required 狀態碼，並附上機器可讀的發票。呼叫 Agent 會自主執行付款，並重新發送附有加密付款證明 Token 的請求。這為按呼叫付費的端點提供了一個標準化的無狀態自動計費選項。

透過解決早期人工智慧生態系中的碎片化問題，A2A 協定建立了一個統一的通訊層，進而讓開發者能在此標準化基礎上進行建構。雖然 A2A 協定核心作為傳輸與協商的骨幹，但實現豐富的交易型應用程式通常需要高度專業化的功能。

此需求透過 A2A Extensions [A2AEXT] 機制解決。它提供了一個標準化模式，讓 Agent 能安全地宣告、協商並執行超越基本訊息傳遞的選用高階功能。

兩個作為 A2A 核心基礎上原生擴充的基礎框架包括：**Agent-to-User Interface (A2UI)**，旨在產生動態、有狀態的使用者體驗；以及 **Universal Commerce Protocol (UCP)**，專為促進安全、自主的 Agent 商務而設計。兩者將在以下章節中詳細討論。

溝通鴻溝

當你問同事「第四季各區域表現如何？」時，他們不會直接丟給你一份試算表。他們會在白板上畫出長條圖，圈出表現突出的區域，並加上註解。這就是人類分享洞察的方式：透過視覺與互動。

然而，我們的 Agent 傳回的仍是原始 JSON。接著你必須自行建立圖表：匯入函式庫、設定座標軸、管理狀態。這種在索取洞察與前端開發之間的上下文切換，會破壞程式碼流暢的開發節奏。

Agent-to-UI (A2UI) 互操作性解決了這個問題，讓 Agent 能夠產生完整且互動的使用者介面作為輸出，而不僅僅是 JSON 資料區塊。

生成式 UI 與 A2UI

什麼是生成式 UI？ 生成式 UI 是指大型語言模型 (LLM) 根據使用者意圖與上下文，在執行階段動態建立使用者介面的概念。開發者不再需要為每一種可能的 UI 狀態進行硬編碼，模型會根據需求即時產生適當的介面。當你詢問「比較各地區第四季銷售額」時，系

統不僅僅是擷取資料，還會組合出互動式版面、卡片、篩選器與控制項，專為該特定請求量身打造。

這代表從靜態、預先建立的介面，轉變為動態、具備上下文感知能力的 UI。挑戰在於如何安全地實現這一點。執行由 LLM 產生的任意 UI 程式碼可能帶來安全風險，例如程式碼注入、XSS 攻擊以及無法控制的副作用。

A2UI：安全的實作方式 A2UI 是一種與框架無關的 UI 意圖宣告標準——Google 開源的一種方式，讓 Agent 能以可攜式、宣告式的格式描述介面，而非傳送原始資料或任意程式碼。

為了理解**格式**在此為何重要，可以想像作曲家如何交付作品。他們不會直接交給音樂家一份錄音，而是交付樂譜。同一份樂譜可以在鋼琴、管弦樂團或合成器上演奏，每種樂器都以自身的聲音詮釋樂譜。A2UI 就是 UI 的樂譜：Agent 撰寫意圖（要渲染什麼以及如何組合），而任何渲染器（例如 React、Angular、Lit、Flutter、Jetpack Compose、SwiftUI 等）都能在使用者實際使用的裝置上原生執行。Agent 不需要知道你的目標平台是網頁、行動裝置、穿戴式裝置還是家電，只需要知道可用元件的目錄以及你想提供的範例。

這種關注點分離正是 A2UI 安全的原因。Agent 不會產生可執程式碼（這是安全上的夢魘），也不會傳送預先渲染的像素（無法重新排版）。

或保持互動模式）。相反地，它會從可信任的目錄中請求元件（例如按鈕、文字欄位、卡片、你自己的圖表），並由用戶端使用自身的元件函式庫進行渲染。目錄定義了可用的元件，Agent 決定如何排列它們，用戶端則組裝最終的結構。這種組合方式如同樂高積木，但積木是來自你的設計系統的 UI 元件。

基本目錄（與自訂目錄） A2UI 提供了一個包含 18 個可立即使用元件的基本目錄，適用於原型開發與展示：

類別	元件
版面配置	Row（列）、Column（欄）、List（清單）
顯示	Text（文字）、Image（圖片）、Icon（圖示）、Divider（分隔線）
容器	Card（卡片）、Modal（彈出視窗）、Tabs（分頁）
媒體	Video（影片）、AudioPlayer（音訊播放器）

類別	元件
互動	Button（按鈕）、TextField（文字欄位）、 CheckBox（核取方塊）、Slider（滑桿）、 DateTimeInput（日期時間輸入）、 ChoicePicker（選擇器）

表 1：A2UI 基本目錄

此套件在 v0.8 版中稱為「standard」，在 v0.9 版中更名為「basic」，這是一個刻意的訊號，表示正式環境的前端應自備目錄，將現有元件（你的設計系統按鈕、圖表、地圖）對應至 A2UI 類型。Agent 本身不會改變，僅渲染器的目錄對應會變動（例如 ChoicePicker 在 v0.8 中為 MultipleChoice）。

以下是 A2UI 訊息的範例（v0.9 格式）：

JSON

```
{
  "version": "v0.9",
  "updateComponents": {
    "surfaceId": "main",
    "components": [
      {"id": "root", "component": "Column", "children": ["title", "summary", "export"]},
      {"id": "title", "component": "Text", "text": "Q4 Sales", "variant": "h1"},
      {"id": "summary", "component": "Text", "text": "Revenue grew 12% QoQ"},
      {"id": "export", "component": "Button", "child": "export-label",
        "action": {"event": {"name": "export_csv"}}},
      {"id": "export-label", "component": "Text", "text": "Export CSV"}
    ]
  }
}
```

程式碼片段 4：A2UI 訊息範例

元件以扁平的鄰接列表形式組織，透過 id 進行參照，這使得結構能夠讓 LLM 逐步產生，並讓客戶端在無需重新渲染的情況下輕鬆更新。另一個 createSurface 訊息（未顯示）會

告知客戶端哪個 id 是根節點。客戶端隨後將其渲染為完整的互動式介面：無需 React 程式碼。

產生 A2UI：兩種模式

實務上有兩種產生 A2UI 的方式，選擇主要取決於版面配置決策的位置。

預設方式是讓 LLM 直接輸出 A2UI：模型掌控版面配置，並根據使用者意圖進行調整，同一個 Agent 可以透過不同的介面處理「比較這些區域」和「顯示趨勢」等需求。此模式的正式程式碼使用官方的 `a2ui-agent-sdk`，詳見第 4.4 節。

特化方式則是讓工具回傳固定的 A2UI 結構：一次工具呼叫，無需消耗 LLM Token 進行 UI 產生，輸出完全可預測。當版面配置可由輸入資料完全決定時，這是最佳選擇：每個區域的銷售儀表板看起來相同，每個預訂表單的欄位也一致。實際上，工具在此扮演伺服器端範本的角色。

一個乾淨的「工具即範本」工具在其主體中會執行兩件事：使用資料繫結（路徑參照，而非 f-string 插值）建立 A2UI 結構，並將其回傳。框架中的 `A2uiPartConverter`（來自 `a2ui-agent-sdk`）會攔截工具的回應，並將其作為 A2UI 部分路由至客戶端，因此工具本身仍保持為普通的 Python 函式：

Python

```
from google.adk.agents import LlmAgent
from google.adk.models import Gemini

def get_sales_dashboard(region: str) -> dict:
    """ 為 `region` 建立資料繫結的銷售儀表板。 """
    data = fetch_sales(region)
    return {
        "version": "v0.9",
        "updateComponents": {
            "surfaceId": "sales",
            "components": [
                {"id": "root", "component": "Column",
                 "children": ["title", "total", "drill"]},
                {"id": "title", "component": "Text",
```

```

        "text": {"path": "/title"}, "variant": "h1"},
        {"id": "total", "component": "Text", "text": {"path": "/total"}},
        {"id": "drill", "component": "Button", "child": "drill-label",
         "action": {"event": {"name": "expand_details"}}},
        {"id": "drill-label", "component": "Text", "text": "Drill Down"},
    ],
},
}

agent = LlmAgent(
    name="sales_agent",
    model=Gemini(model="gemini-flash-latest"),
    tools=[get_sales_dashboard],
)

# 在執行器設定時連接轉換器，使此工具的回應成為 A2UI 部分：

# from a2ui.adk.send_a2ui_to_client_toolset import A2uiPartConverter

# A2aAgentExecutorConfig(event_converter=A2uiPartConverter(catalog,

#                          bypass_tool_check=True))

```

程式碼片段 5：LLM 生成 UI 模式

資料值透過平行的 `updateDataModel` 訊息流動，該訊息會解析 `{path: "/title"}` 的參照，因此客戶端可以在資料更新時重新渲染，而無需重新傳送結構。LLM 只會看到結構化的工具回應（而非渲染後的 UI），因此其上下文能專注於下一步該做什麼，而非剛剛建立的 UI。

何時使用各模式	回傳
查詢	回傳
「平均值是多少？」	資料（文字）
「比較這些區域」	由 LLM 生成的 UI (§4.4)
「顯示我的儀表板」	由工具建立的 UI（本節）

何時使用各模式	回傳
API 對 API	資料 (JSON)

表 2：查詢與輸出對應

當互動或視覺化能為原始資料增值時，使用 A2UI，並根據佈局決策的擁有者選擇模式：由 LLM（意圖驅動）或確定性模板（輸入驅動）決定。

互動式產物與 Canvas

傳統的聊天介面是線性的，每個回應都是靜態的。Canvas 方法則是

不同之處在於：它建立了一個持久化的工作區，讓 Agent 和使用者都能進行編輯。

你不再需要滾動瀏覽對話紀錄，而是擁有一份活文件。Agent 可以修改特定區段，你也可以手動編輯，雙方的變更都會即時反映。

Canvas + A2UI 將 Canvas 的持久化功能與 A2UI 的互動性結合，能建立實用的工作流程：

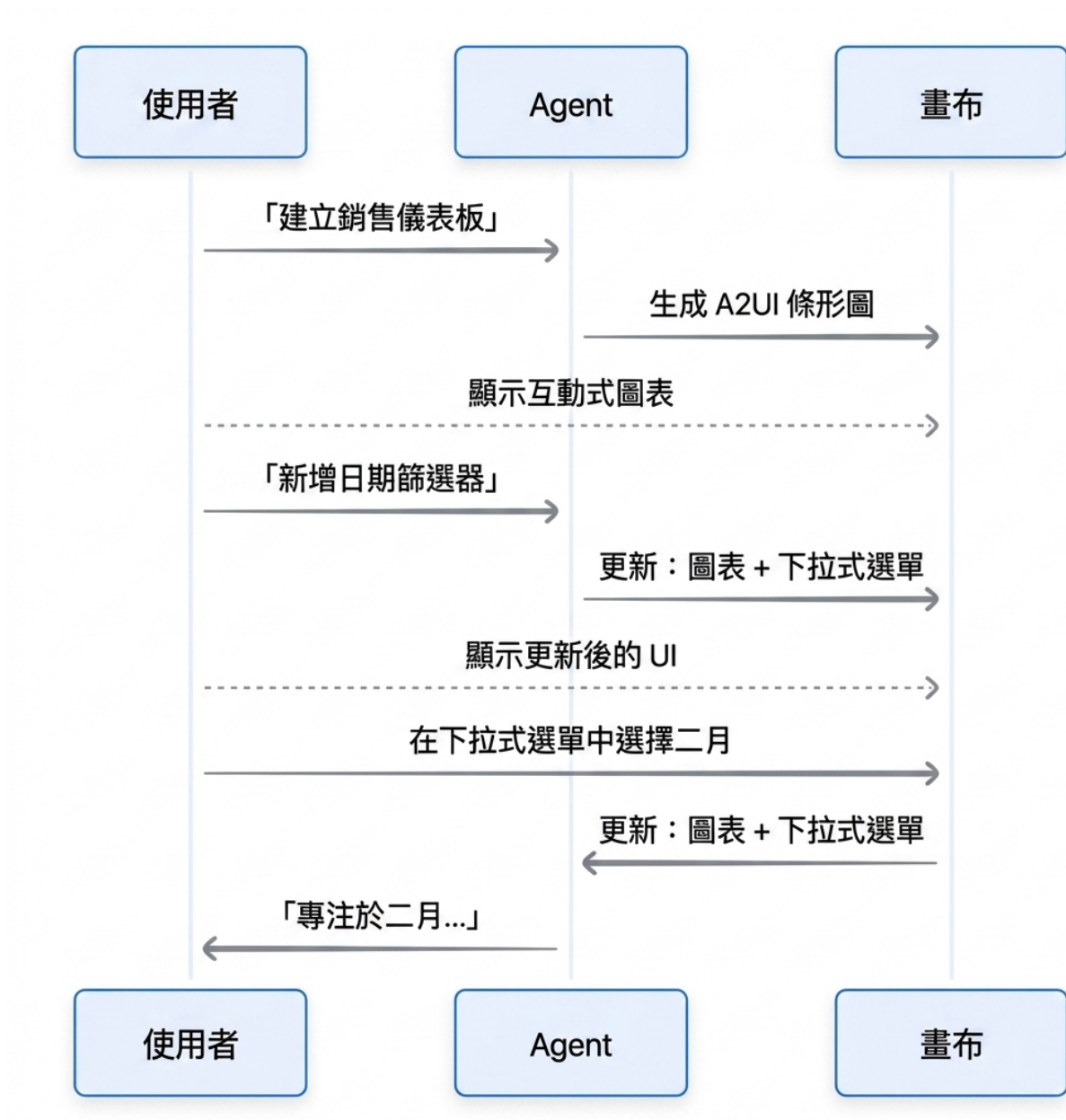


圖 7：使用者、Agent、Canvas 互動流程

UI 不僅僅是渲染出來的介面，它是一種溝通媒介。Agent 會觀察你的互動操作並做出相應回應。

最佳實踐

讓 LLM 生成 A2UI 對於 LLM 生成 UI 的模式（4.2 版本引入的預設模式），手動編寫 A2UI JSON 既繁瑣又耗時。建議使用官方的 `a2ui-agent-sdk` (`pip install a2ui-agent-`

sdk)：A2uiSchemaManager 會建立一個系統 Prompt，其中已嵌入目錄結構描述 (schema) 和範例，目錄本身也提供 JSON-Schema 驗證器，而同一個 SDK 還包含解析 Agent 輸出的 <a2ui-json> 區塊的解析器。接著，你可以在驗證時遇到結構描述錯誤時重試。

Python

```
# pip install a2ui-agent-sdk google-adk
import asyncio, json
import jsonschema
from a2ui.schema.manager import A2uiSchemaManager
from a2ui.basic_catalog.provider import BasicCatalog
from a2ui.schema.constants import VERSION_0_9
from a2ui.parser.parser import parse_response
from google.adk.agents import LlmAgent
from google.adk.models import Gemini
from google.adk.runners import InMemoryRunner
from google.genai import types

# 1. SDK 載入目錄、建立驗證器，並產生一個完整的系統 Prompt，
# 其中已內嵌結構描述與範例。
schema_manager = A2uiSchemaManager(
    version=VERSION_0_9,
    catalogs=[BasicCatalog.get_config(version=VERSION_0_9)],
)
catalog = schema_manager.get_selected_catalog()

agent = LlmAgent(
    model=Gemini(model="gemini-flash-latest"),
    name="ui_agent",
    instruction=schema_manager.generate_system_prompt(
        role_description=" 你產生互動式 UI 為 A2UI v0.9 訊息。",
        ui_description=" 使用 Cards、Lists、ChoicePickers 和 Buttons 來呈現資料。",
        include_schema=True,
```

```

        include_examples=True,
    ),
)

# 2. 執行 Agent。它會輸出包含一個或多個 <a2ui-json>...</a2ui-json>

# 區塊的文字；parse_response() 會解析出 JSON。驗證後，若有錯誤則重試。
async def create_ui(intent: str, data: dict, max_retries: int = 3) -> list:
    runner = InMemoryRunner(agent=agent, app_name="ui_demo")
    session = await runner.session_service.create_session(
        app_name="ui_demo", user_id="u",
        state={"expression": "{expression}"}, # 跳脫 SDK 的樣板預留位置
    )
    query = f"{intent}"

    Data: {json.dumps(data)}"
    last_error = None
    for _ in range(max_retries + 1):
        chunks = []
        msg = types.Content(role="user", parts=[types.Part(text=query)])
        async for ev in runner.run_async(user_id="u",
                                          session_id=session.id, new_message=msg):
            if ev.content and ev.content.parts:
                chunks.extend(p.text for p in ev.content.parts if p.text)
        blocks = [rp.a2ui_json for rp in parse_response("".join(chunks))
                  if rp.a2ui_json]
        try:
            for b in blocks:
                for m in (b if isinstance(b, list) else [b]):
                    catalog.validator.validate(m)
            return blocks
        except jsonschema.ValidationError as e:
            last_error = e
            path = "/".join(str(p) for p in e.absolute_path)

```

```

        query = (
            f" 你之前的回應在 {path} 驗證失敗："
            f"{e.message[:200]}\n修正並重試："
            f"{intent}\nData: {json.dumps(data)}"
        )
        raise ValueError(f" 經過 {max_retries} 次重試後，結構描述驗證仍失敗：{last_error}")

```

程式碼片段 6：A2uiSchemaManager 實作

在正式環境中，將 `create_ui()` 包裹在 `try/except` 中，並在結構驗證失敗時回退至文字回應。LLM 輸出具有隨機性，渲染器絕不應接收到格式錯誤的負載。

混合輸出以提升彈性 同時提供資料與 UI，讓使用者可自行選擇：

JSON

```

{
  "data": {"sales": [...]},
  "ui": {"version": "v0.9", "updateComponents": {"surfaceId": "main",
  "components": [...]}},
  "ui_available": true
}

```

程式碼片段 7：混合輸出範例結構

API 用戶端會忽略 `ui` 欄位並使用 `data`。面向人類的用戶端則會渲染 A2UI 訊息。

摘要

生成式 UI 讓 LLM 能根據使用者意圖在執行階段建立使用者介面。A2UI 是 Google 開源的安全標準，提供與框架無關的格式來宣告 UI 意圖，讓同一個 Agent 訊息能在 Lit、Flutter、React 或你自己的設計系統中原生渲染。

安全模型至關重要：Agent 無法注入任意程式碼，只能請求渲染器目錄中已受信任的元件。這正是 A2UI 在實現動態生成式介面的同時仍能保持安全的原因。

Agent 與商務 (AP2 與 UCP)

具備工作流程範例的自主採購

Vibe 開發者透過其 Agent 學習結帳、目錄、訂單功能及授權流程。

協定的主要特性與優勢

具型別的結構描述、安全性與開源（整合負債與供應商中立性）

Lab recommendations:

<https://codelabs.developers.google.com/next26/adk-agent-commerce#0>

雖然前面的章節著重於探討程式碼 Agent 在 Agentic 架構中如何與工具、其他 Agent 以及使用者介面進行互動，但這些互動通常僅限於使用 MCP、A2A 和 A2UI 等協定進行「讀取」操作。隨著這些系統的演進，Agent 必須逐步轉向執行具有實際財務影響的「動作」。

優先考量商務協定並建立穩健的操作框架，能確保你的 Agent 在處理交易時嚴格遵循產業標準。

為了理解如何實作這些系統，我們必須先探討這些協定的本質及其互補角色。

AP2 與 UCP 是什麼？

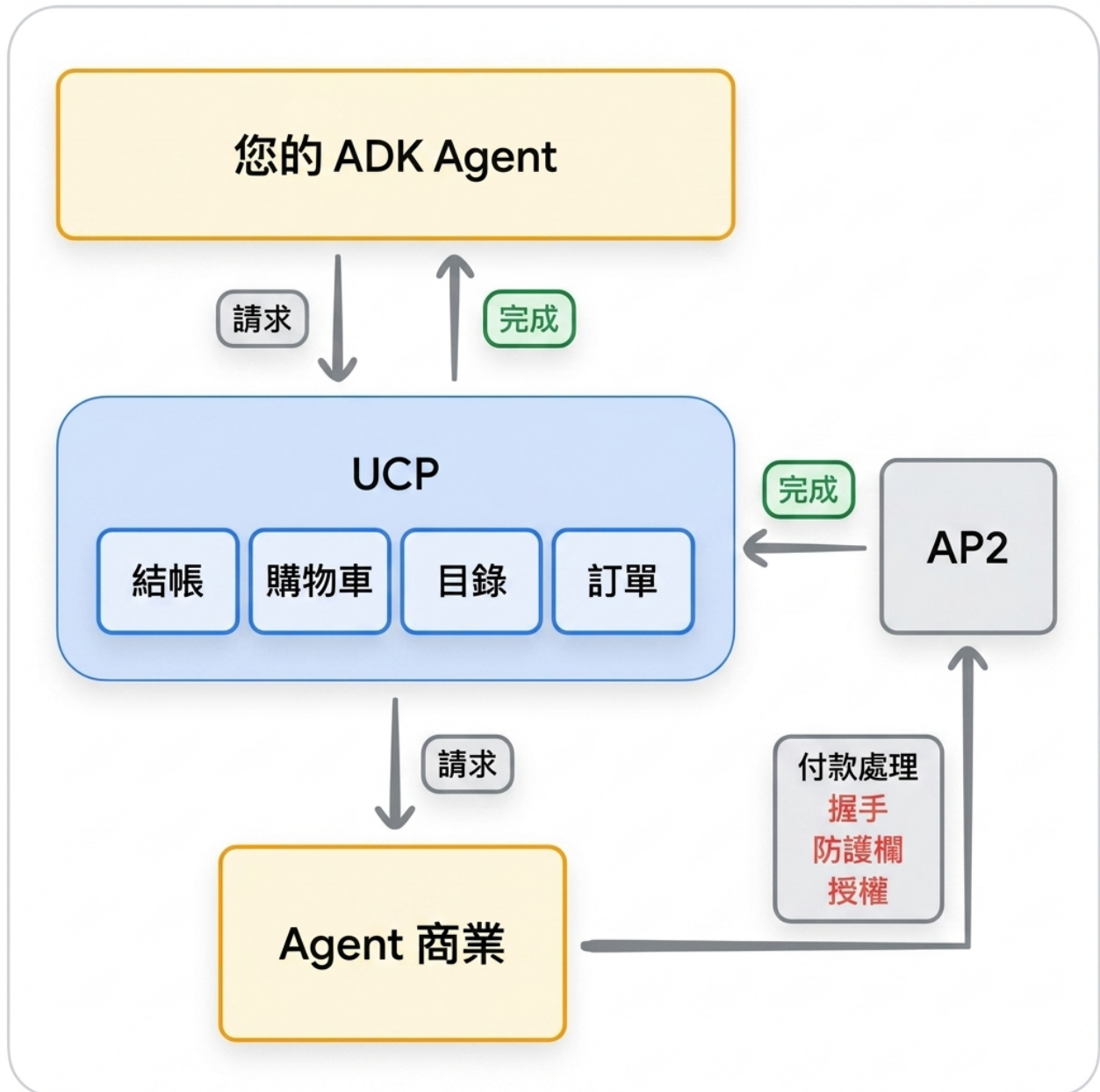


圖 8：AP2 與 UCP 生態系

想像你和室友在凌晨 2 點餓到不行，但你忙著讀書無法停下，於是你部署 AI 智慧助理為公寓訂餐。以下是 UCP (Universal Commerce Protocol, 通用商務協定) 和 AP2 (Agent Payments Protocol, Agent 支付協定) 如何分工，將墨西哥捲餅送到宿舍大廳。

UCP (Universal Commerce Protocol)：終極外送應用程式 在 2024 年，你的 AI 必須手動開啟 Chrome、進入當地墨西哥餐廳設計不良的網站、嘗試點選「加酪梨醬」的核取方塊，並祈禱網站不會當機。而 UCP 則像是一個通用翻譯器，校園內的每家餐廳都以乾淨、標準的機器語言發布菜單、營業時間和客製化選項。

你的 AI 透過 UCP 向餐廳系統詢問：「你們還在營業嗎？有素食墨西哥捲餅嗎？」若要下單，可以詢問：「一份雞肉墨西哥捲餅，不要洋蔥，加酸奶油，再加一份薯片」，餐廳系統則會回覆稅金、外送費和預計送達時間。

簡而言之，UCP 是你的 AI 與商店溝通、查看選項並組合完美訂單的方式。

在 UCP 之前，你的應用程式能夠進行探索並取得資訊。然而，這些不同提供者之間的互動需要個別進行協調。透過標準化互動方式，我們讓非人類實體之間的互動變得更加容易。

AP2 (Agent 支付協定)：父母的信用卡，但有 嚴格規則 現在食物已經放進購物車，但你的 AI 需要付款。你顯然不會在 AI 的 Prompt 中輸入實際的簽帳金融卡號碼，然後說「隨便刷」。

AP2 是一個開放、共享的協定，為 Agent 與商家之間的安全、合規交易提供共通語言，讓你的 AI 只能在你設定的規則範圍內支付食物費用。

- **防護欄（強制規則）**：在你讓 AI 執行任務之前，你會核准一條數位規則：「你最多可以在 Taco Bell 花費 25 美元。」
- **握手協議**：當 AI 結帳時，它不會顯示你的卡號，而是顯示一個由你簽署的數位加密「承諾支付」憑證，上面寫著：「我的使用者核准了這筆 18.50 美元的訂單。」餐廳的銀行會即時驗證此數位簽章。
- **無隱藏費用**：如果餐廳試圖偷偷向你收取 50 美元而非 18.50 美元，AP2 協定會立即阻止，因為這違反了你核准的規則。

總結：AP2 是一個安全的保險箱，讓你的 AI 能夠使用你的資金支付費用，但確保它不會不小心買下一台 1,000 美元的電視。

UCP	AP2
決定購買內容、處理菜單並將食物放入購物車的大腦。	如同錢包，安全處理付款流程，避免你被詐騙。
整合任何商業提供者	整合支付

UCP	AP2
統一整合：共享語言：可擴充架構安全優先方法	授權與可稽核性意圖真實性 Agent 錯誤與幻覺可追溯性

表 3：UCP 與 AP2 比較

- 如果你是開發者，正在為商家或支付處理器建立 UCP Agent，請依照這裡的指示操作。
- 如果你是開發者，正在為商家或付款處理業者建立 AP2 Agent，請依照這裡的指示操作。
- 如果你是開發者，正在建立一個消費 AP2 Agent 的 Agent（例如購物者），請依照這裡的指示操作。

結論

透過採用 MCP、A2A、A2UI、AP2 和 UCP 等基礎標準，組織能夠消除客製化整合所帶來的沉重技術債務，並將重心完全放在編排高價值的業務邏輯上。這一範式轉變，從根本上將開發者從僅僅連接脆弱 API 的技工，提升為全球自主化工作力的真正架構師。隨著這些標準化通訊層的成熟，它們將釋放全新的規模經濟，徹底改變企業軟體的建構、消費與貨幣化方式。

註釋

1. A2A Community (2026) A2A Documentation: Extensions, A2A Protocol, <https://a2a-protocol.org/latest/topics/extensions/>
2. Antigravity Team (2026) Antigravity Editor: MCP Integration, Antigravity Docs, <https://antigravity.google/docs/mcp>
3. Fowler M, Lewis J (2014) Microservices, MartinFowler, <https://martinfowler.com/articles/microservices.html>
4. Google A2A Team (2025) Announcing the Agent2Agent Protocol (A2A), Google Developers Blog, <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>

5. Google A2UI Team (2025) Introducing A2UI: An Open Project for Agent-Driven Interfaces, Google Developers Blog, <https://developers.googleblog.com/introducing-a2ui-an-open-project-for-agent-driven-interfaces/>
6. Google A2UI Team (2026) A2UI v0.9: The New Standard for Portable, Framework-Agnostic Generative UI, Google Developers Blog, <https://developers.googleblog.com/a2ui-v0-9-generative-ui/>
7. Google A2UI Team (2026) A2UI [電腦軟體], GitHub, <https://github.com/google/A2UI>
8. Google Cloud Databases Team (2025) MCP Toolbox for Databases : 簡化 AI Agent 存取企業資料, Google Cloud Blog, <https://cloud.google.com/blog/products/ai-machine-learning/mcp-toolbox-for-databases-now-supports-model-context-protocol>
9. Google Cloud Team (2026) 發表 Agents-to-Payments (AP2) Protocol, Google Cloud Blog, <https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol?e=48754805>
10. Google Cloud Team (2026) 在 AI 應用程式中設定 MCP, Cloud Docs, <https://docs.cloud.google.com/mcp/configure-mcp-ai-application#antigravity>
11. Google Stitch Team (2026) Google Stitch 文件 : MCP 設定, Stitch with Google Docs, <https://stitch.withgoogle.com/docs/mcp/setup>
12. MCP Team (2026) Model Context Protocol 規格書, MCP Docs, <https://modelcontextprotocol.io/specification/2025-11-25>
13. Muchandi V (2026) Rad-skills : 用於快速 Agent 開發的 Gemini 技能, GitHub, <https://github.com/VeerMuchandi/rad-skills>
14. Hotz H (2026) Agentic Commerce 革命, O'Reilly Radar, <https://www.oreilly.com/radar/the-agentic-commerce-revolution/>
15. Pichai S (2017) 讓人工智慧為每個人服務, Google Blog, <https://blog.google/innovation-and-ai/products/making-ai-work-for-everyone/>
16. Saboo S, Overholt K (2026) AI Agent 協定開發者指南, Google Developers Blog, <https://developers.googleblog.com/developers-guide-to-ai-agent-protocols/>
17. Styer M, Patlolla K, Mohan M, Diaz S (2025) 使用 Model Context Protocol (MCP)

實現 Agent 工具與互通性, Kaggle, <https://www.kaggle.com/whitepaper-agent-tools-and-interoperability-with-mcp>

18. The Linux Foundation (2026) A2A Protocol 突破 150 個組織，進駐主要雲端平台，並在第一年實現企業生產應用, Linux Foundation Press, <https://www.linuxfoundation.org/press/a2a-protocol-surpasses-150-organizations-lands-in-major-cloud-platforms-and-sees-enterprise-production-use-in-first-year>

19. 通用 AI 目錄與登錄標準文件 (2026) ,

<https://github.com/Agent-Card/ai-catalog>

20. Ricardo Cataldi (2026) , 《MCP vs A2A : 工具、Agent 與各協定的適用場景》, <https://levelup.gitconnected.com/mcp-vs-a2a-tools-agents-and-where-each-protocol-belongs-53e1f9ab9765>

21. Lukas Geiger (2026) , 《AI Agent 變現：確保獲利能力》,

<https://medium.com/google-cloud/monetizing-ai-agents-ensuring-profitability-92efa535ebb3>